

PMA Technisches Briefing

PMA ist technisch als klassische Webanwendung aufgebaut: ein React-Frontend kommuniziert per REST mit einem Node.js-/Express-Backend, das seine Daten in MongoDB speichert. Für Orts- und Stadtvorschläge wird zusätzlich die externe Nominatim-/OSM-Suche angebunden.

Verwendete Technologien

- **Frontend:** React, React Router, Tailwind CSS, Leaflet, React Leaflet
- **Backend:** Node.js, Express
- **Datenbank:** MongoDB mit Mongoose
- **Tests:** `node --test` mit `mongodb-memory-server`
- **Externe API:** Nominatim / OpenStreetMap für Ortsvorschläge

Frontend-Aufbau

Das Frontend ist als Single-Page-Application umgesetzt. React Router verwaltet die Seitenübergänge zwischen Home, Login, Registrierung, Profil, Swipe, Matches, Chat, Events und Kartenansicht. Geschützte Bereiche werden nur angezeigt, wenn ein gültiger Login-Status vorliegt.

Die Authentifizierung wird zentral über einen React Context verwaltet. Das Session-Token liegt im `localStorage`. Beim Start der App wird über einen API-Aufruf geprüft, ob das Token noch gültig ist; dadurch kann die Session automatisch wiederhergestellt werden.

Für die Kartenansicht werden `leaflet` und `react-leaflet` eingesetzt. Das Styling kombiniert Tailwind-Klassen mit normalem CSS.

Backend-Aufbau

Das Backend ist als Express-Monolith aufgebaut und stellt seine REST-Schnittstellen unter dem Prefix `/api/v1` bereit. Die API ist in fachliche Module aufgeteilt:

- **auth:** Registrierung, Login, Session-Token
- **users:** Profilpflege, öffentliche Profilansicht
- **discovery:** Datenbasis für den Swipe-Bereich

- **matches:** Like/Pass-Logik und Match-Erkennung
- **chat:** Nachrichten zwischen gematchten Nutzern
- **events:** Erstellen, Bearbeiten, Löschen und Filtern von Events
- **map:** Kartenübersicht mit Events und Nutzerzonen
- **places:** Orts- und Stadtvorschläge über OSM

Intern folgt das Backend grob dem Muster `routes -> controller -> service -> model`. Dadurch sind HTTP-Handling, Fachlogik und Datenzugriff sauber getrennt.

Datenhaltung

Die Persistenz läuft über MongoDB. Mongoose-Modelle definieren die wichtigsten Collections:

- **User:** Kontodaten, Profil, Präferenzen, Bilder, Status
- **Session:** Login-Sessions mit Bearer-Token
- **Swipe:** Likes und Passes
- **Message:** Chatnachrichten
- **Event:** Eventdaten inklusive Position und Ersteller

Bilder werden aktuell noch nicht als echte Dateien hochgeladen, sondern als Data-URLs im JSON gespeichert. Das spart Infrastruktur, ist aber klar eine MVP-Vereinfachung.

Wie die Teile verbunden sind

1. Ein Nutzer registriert sich oder loggt sich im React-Frontend ein.
2. Das Frontend sendet die Daten per HTTP an das Express-Backend.
3. Das Backend erzeugt bei erfolgreichem Login ein Session-Token und speichert es in MongoDB.
4. Das Frontend speichert dieses Token lokal und sendet es bei geschützten Requests als `Authorization: Bearer ...` mit.
5. Die API prüft das Token und ordnet den Request einem Nutzer zu.
6. Die jeweiligen Fachmodule lesen oder schreiben dann Daten in MongoDB.
7. Für Ortsvorschläge fragt das `places`-Modul zusätzlich Nominatim/OSM ab.

Wichtige Architekturentscheidungen

- **Monolith statt Microservices:** weniger Overhead, leichter testbar und für einen MVP schneller umzusetzen
- **REST statt WebSockets:** der Chat ist anfragebasiert und bewusst noch nicht in Echtzeit umgesetzt
- **MongoDB statt relationaler Datenbank:** flexibel für profilspezifische, unterschiedlich strukturierte Nutzerdaten
- **Data-URL-Bilder statt Dateispeicher:** pragmatische MVP-Lösung mit klaren Grenzen

Fazit

Die technische Architektur ist bewusst simpel, aber sauber strukturiert. Sie reicht für einen funktionierenden MVP mit Registrierung, Swipe-Flow, Matches, Chat, Events und Karte. Gleichzeitig ist gut erkennbar, wo spätere Ausbaustufen liegen würden, etwa bei Echtzeit-Chat, echtem Datei-Upload oder produktionsreifem Deployment.